

ROS Control

ROS Control

1. Course Content
2. Preparations
 - 2.1 Content Description
 - 2.2 Start the Agent
3. Launch Commands
 - 3.1 Function Description
 - 3.2 Program Start
 - 3.2.1 View Node Information
 - 3.2.2 Publish Control Commands
 - 3.2.3 Control the Car's Buzzer
 - 3.2.4 Control the Car's Light Strip
 - 3.2.5 Subscribe to LiDAR Data
 - 3.2.6 Subscribe to Battery Level Data
 - 3.2.7 Subscribe to IMU Data
 - 3.2.8 Subscribe to Odometry Data

1. Course Content

1. Learn the basic method of controlling a robot via ROS.

This function allows for controlling the car's speed and buzzer using the `ros2 topic` tool, and also for reading low-level data such as LiDAR data, IMU data, odometry data, etc.

2. Preparations

2.1 Content Description

This lesson uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host machine, enter the command to get into the Docker container, and then input the commands mentioned in this lesson in the terminal within the Docker container. A tutorial on how to enter the Docker container from the host can be found in this product's guide [00.Configuration and Operation Guide]-[3.Enter the Docker (For RPI5)]. For Orin and RDKX5 boards, you can directly open a terminal and enter the commands mentioned in this lesson.

2.2 Start the Agent

Note: To test all examples, you must start the Docker agent first. If it is already running, you do not need to start it again. The agent starts by default on boot, so there is no need to restart it.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~  
jetson@yahboom: ~ 177x26  
jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Launch Commands

3.1 Function Description

This function allows for controlling the car's speed and buzzer using the `ros2 topic` tool, and also for reading low-level data such as LiDAR data, IMU data, odometry data, etc.

3.2 Program Start

3.2.1 View Node Information

After successfully connecting to the agent, you can enter `ros2 node list` in the terminal to see which nodes are available.

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Enter the command,

```
ros2 node list
```

The execution screenshot is as follows,

```
Jetson@yahboom: ~  
jetson@yahboom: ~ 80x30  
jetson@yahboom:~$ ros2 node list  
/YB_Node
```

Here, `/YB_Node` indicates that the low-level control node has started. Enter the following command in the terminal to query information about this node,

```
ros2 node info /YB_Node
```

The execution screenshot is as follows,

```
/YB_Node
jetson@yahboom:~$ ros2 node info /YB_Node
/YB_Node
Subscribers:
  /beep: std_msgs/msg/UInt16
  /cmd_vel: geometry_msgs/msg/Twist
  /rgb: std_msgs/msg/ColorRGBA
Publishers:
  /battery: std_msgs/msg/Float32
  /imu/data_raw: sensor_msgs/msg/Imu
  /odom_raw: nav_msgs/msg/Odometry
  /scan2: sensor_msgs/msg/LaserScan
  /scan3: sensor_msgs/msg/LaserScan
Service Servers:

Service Clients:

Action Servers:

Action Clients:
```

A table has been compiled here to show which topics the node publishes and subscribes to, along with the message data type for each topic. The table is as follows,

Subscription Topics Table

Subscribed Topic	Topic Message Type	Topic Attribute
/beep	std_msgs/msg/UInt16	Control Buzzer
/cmd_vel	geometry_msgs/msg/Twist	Control Car Speed
/rgb	std_msgs/msg/ColorRGBA	Control Light

Publication Topics Table

Published Topic	Topic Message Type	Topic Attribute
/battery	std_msgs/msg/Float32	Publish Battery Level Data
/imu/data_raw	sensor_msgs/msg/Imu	Publish IMU Data
/odom_raw	nav_msgs/msg/Odometry	Publish Odometry Data

LiDAR Topics for Dual-LiDAR Version

Published Topic	Topic Message Type	Topic Attribute
/scan0	sensor_msgs/msg/LaserScan	Left-Rear LiDAR Data of the Car
/scan1	sensor_msgs/msg/LaserScan	Right-Front LiDAR Data of the Car

LiDAR Topics for Single-LiDAR Version

Published Topic	Topic Message Type	Topic Attribute
/scan2	sensor_msgs/msg/LaserScan	LiDAR First-Half Scan Data
/scan3	sensor_msgs/msg/LaserScan	LiDAR Second-Half Scan Data

3.2.2 Publish Control Commands

Based on the subscription topics table, use the command format `ros2 topic pub <topic_name> <message_type> <data> --once` to publish a single frame of control data.

For the first test, it is recommended to lift the car so that its wheels are off the ground. We will make the car move forward at a linear velocity of 0.1m/s. Enter the following command in the terminal,

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" --once
```

After execution, the car will move forward at a speed of 0.1m/s. Similarly, to control the car to rotate at an angular velocity of 1.0rad/s, assign a value to the angular z component. The command is as follows,

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.0}}" --once
```

After execution, the car will rotate. If we want to stop the car, we can publish a message with both linear and angular velocities set to 0. The command is as follows,

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" --once
```

The `--once` flag means that only one frame of the message is sent. For other parameters of `ros2 topic pub`, please refer to this product's course [15. ROS2 Basic Course] under [19. Common ROS2 Command Tools].

3.2.3 Control the Car's Buzzer

To activate the buzzer, enter the following command in the terminal,

```
ros2 topic pub /beep std_msgs/msg/UInt16 "data: 1" --once
```

To deactivate the buzzer, enter the following command in the terminal,

```
ros2 topic pub /beep std_msgs/msg/UInt16 "data: 0" --once
```

3.2.4 Control the Car's Light Strip

To publish the color red, enter the following command in the terminal,

```
ros2 topic pub /rgb std_msgs/msg/ColorRGBA "{r: 1.0, g: 0.0, b: 0.0, a: 1.0}" --once
```

Parameters related to `a`:

- a: 101.0 Flowing light
- a: 102.0 Marquee light
- a: 103.0 Breathing light
- a: 104.0 Gradient light
- a: 105.0 Starlight effect
- a: 106.0 Battery level display

Based on the publication table, use the command format `ros2 topic echo <topic_name>` to receive sensor data published by the car's node.

3.2.5 Subscribe to LiDAR Data

Dual-LiDAR version: The topic name for the left-rear LiDAR is `/scan0`, and for the right-front LiDAR is `/scan1`. Here we use the right-front LiDAR data as an example.

Single-LiDAR version: The complete data for one scan of the LiDAR is published on two topics, `/scan2` and `/scan3`, with each topic representing half a scan.

- Here is an example using the `/scan1` topic from the dual-LiDAR version:

```
ros2 topic echo /scan1
```

The subscribed data is shown in the figure below:

```
header:
  stamp:
    sec: 1749725466
    nanosec: 564000000
  frame_id: laser1_frame
angle_min: 0.0
angle_max: 6.2831854820251465
angle_increment: -0.009442231617867947
time_increment: 0.0
scan_time: 0.0
range_min: 0.05000000074505806
range_max: 12.0
ranges:
- 0.5
- 0.5019999742507935
- 0.5139999985694885
- 0.5189999938011169
- 0.5239999890327454
- 0.5389999747276306
- 0.5429999828338623
- 0.5659999847412109
- 0.5899999737739563
- 0.8090000152587891
- 0.8610000014305115
- 0.859000027179718
```

3.2.6 Subscribe to Battery Level Data

The normal operating voltage of this product is theoretically between 9.6V and 12V. If the voltage drops below 10.3V, the buzzer will beep to indicate a low battery, and the battery needs to be charged. You can enter the following command in the terminal to check the battery voltage:

[!CAUTION]

Special Note!!!: For lithium batteries, the relationship between voltage and charge level is not linear. It is not always the case that a lower voltage means a lower charge. When the vehicle's alarm sounds, it means the battery level is already too low and needs to be charged promptly.

- Over-discharging lithium batteries is prohibited, as long-term over-discharging can cause the battery to swell and become damaged.

```
ros2 topic echo /battery
```

The subscribed data is shown in the figure below:

```
data: 11.82494068145752
---
data: 11.815200805664062
---
data: 11.828187942504883
---
data: 11.828187942504883
---
data: 11.821694374084473
---
data: 11.818448066711426
---
data: 11.828187942504883
---
data: 11.821694374084473
---
data: 11.818448066711426
---
data: 11.828187942504883
---
data: 11.828187942504883
---
data: 11.818448066711426
---
data: 11.837928771972656
---
```

This indicates that the battery voltage is 11.8V.

3.2.7 Subscribe to IMU Data

The control board is equipped with a 9-axis IMU sensor that can provide feedback on the car's attitude. Enter the following command in the terminal to get the IMU data,

```
ros2 topic echo /imu/data_raw
```

The subscribed IMU data is shown in the figure below:

```
header:
  stamp:
    sec: 1749726055
    nanosec: 333000000
    frame_id: imu_frame
orientation:
  x: 0.004843415226787329
  y: 0.0012736533535644412
  z: -0.0036820757668465376
  w: 1.0
orientation_covariance:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
angular_velocity:
  x: -0.0028961878269910812
  y: -0.0014647386269643903
  z: -0.005825664848089218
angular_velocity_covariance:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
linear_acceleration:
  x: -0.08140286058187485
  y: 0.018555063754320145
```

3.2.8 Subscribe to Odometry Data

The four motors of this product are equipped with encoders. The ROS control board reads the encoder information and then publishes the calculated odometry data. Enter the following command in the terminal to read the odometry data,

```
ros2 topic echo /odom_raw
```

The subscribed odometry data is shown in the figure below,

